

Δ KV: PATCHING THE KV CACHE WHEN WEIGHTS CHANGE

Anonymous authors

Paper under double-blind review

ABSTRACT

Every production LLM serving stack treats a weight update as a cache miss. vLLM namespaces prefix blocks by LoRA name, SGLang isolates by `weight_version`, and RL trainers flush every KV byte on every policy step. The universal algorithm is: *weights changed* $\rightarrow$ *throw away all computed state* $\rightarrow$ *recompute from scratch*. We argue this is the wrong primitive. The KV cache is a materialized view over (weights $\times$ tokens); a weight update should trigger incremental view maintenance, not invalidation. We present Δ KV, a library that, given a compact ΔW (LoRA swap, ROME edit, quantization change, or KL-bounded RL step), computes the corresponding Δ KV at a fraction of re-prefill cost. The cache key is the token prefix only: adapter identity and weight version are edges in a lineage graph, not part of the key. At any layer whose input activations are stored, the projection patch $\Delta K = \text{RoPE}(X \Delta W_K^\top)$, $\Delta V = X \Delta W_V^\top$ is *exact* because RoPE is linear. Deeper layers are maintained by a hybrid of (i) the adapter path on cached hidden states, (ii) rank- r least-squares onto the LoRA column space plus an orthogonal residual shift, (iii) first-order Jacobian propagation of ΔX with periodic snapshot resets, and (iv) a condition-number-anchored probe budget. A compatibility lattice strict $\rightarrow$ patched- ε $\rightarrow$ miss makes the system pointwise never worse than today’s flush-and-recompute. On a CPU Llama-style reference stack, a rank-4 LoRA of scale 10^{-2} is patched with next-token KL ≈ 0 , relative KV $\ell_2 \approx 2 \times 10^{-3}$ (inside a 4-bit KIVI noise floor), and 8.5% of re-prefill FLOPs. Δ KV does not fork vLLM or SGLang; it plugs into the connectors those engines already expose.

1 INTRODUCTION

Prefix caching is the main reason LLM serving is fast. vLLM’s paged attention (Kwon et al., 2023) and SGLang’s radix tree (Zheng et al., 2024) reuse KV for shared token prefixes across requests, LoRA adapters, and RL rollouts. That reuse has two well-understood axes: *tokens* (the prefix) and *position* (RoPE). A third axis is almost always ignored: *the weights*.

Whenever the weights move—loading a LoRA (Hu et al., 2022), swapping adapters, applying a ROME write (Meng et al., 2022), changing a quantization grid, or taking a PPO/GRPO step—today’s engines declare the entire KV cache invalid. vLLM hashes `lora_name` into the prefix-block key and, on a same-name reload, silently reuses stale blocks.¹ SGLang puts `weight_version` in the radix `extra_key` (SGLang Developers, 2025) and flushes HiCache on every update. veRL and Mooncake hard-reset every KV byte on every policy step. The result is that the most expensive tensor in the system is thrown away the moment it becomes slightly wrong, even when ΔW is *small by construction* (rank- r LoRA, rank-1 edit, KL-bounded RL).

This paper makes a simple claim: **KV is a materialized view over (weights $\times$ tokens); Δ KV is incremental view maintenance under weight updates.** Database engines have done this for decades. Serving engines have not, because they treat the cache as an opaque byte buffer keyed by a hash that includes the weight identity.

¹<https://github.com/vllm-project/vllm/issues/42125>; RFC <https://github.com/vllm-project/vllm/issues/48312>.

We introduce Δ KV, a product-shaped library that replaces invalidation with maintenance. A weight update ships a compact *delta descriptor* (PEFT factors, a ROME outer product, or an SVD sketch). The next lookup of a cached prefix materializes a *patched view* instead of missing. If a conservative error estimate exceeds a budget ε , the engine does exactly what it does today: recompute. You cannot do worse than flush.

Contributions.

1. **A cache contract** in which keys are token prefixes only and compatibility is a lattice strict $\rightarrow$ patched- $\varepsilon \rightarrow$ miss (§2).
2. **Exact projection patches.** Because RoPE is linear, $\text{RoPE}(K + \Delta K) = \text{RoPE}(K) + \text{RoPE}(\Delta K)$. At any layer whose input X is stored, ΔK and ΔV are closed-form and numerically exact (§4).
3. **A rank- r LoRA-subspace corrector** that generalizes AgentKVShift’s rank-1 mean-shift μ to the column space of B , with an orthogonal residual and a CacheBlend-style residual gate (§5.1).
4. **LoRC-style κ -anchored hybrid routing:** per-layer skip / exact / subspace / probe planned from cumulative condition numbers, plus first-order ΔX propagation with snapshot resets (§5.3–§5.2).
5. **Connectors, not forks:** vLLM KVConnectorBase_V1, SGLang update_weights_*, LMCache HiddenStateStore, PEFT state-dict extraction (§6).
6. **A kill-criterion validator** on a real Llama-style stack (RMSNorm, RoPE, GQA, SwiGLU) that reports next-token KL, log-prob L_1 , and a KIVI-4bit floor—not KV- ℓ_2 alone (§7).

2 SETUP AND CACHE CONTRACT

A decoder-only Transformer maps a token prefix $x_{1:n}$ and weights W to a KV cache $C = \text{KV}(W, x_{1:n}) \in \mathbb{R}^{L \times n \times n_{\text{kv}} \times d_h}$ and a next-token distribution $\pi(\cdot \mid x_{1:n}; W, C)$. A *weight version* v is a node in a lineage DAG. A *weight delta* $\Delta W_{v \rightarrow v'}$ is a compact descriptor of $W_{v'} - W_v$ (§3). The serving question is: given C computed under v , produce a cache C' that is a valid view under v' *without* running a full prefill.

Entry. A cache row is

$$\text{CacheEntry} = \left(\underbrace{C_{\text{base}}}_{\text{KV under } v_0}, \underbrace{(\Delta C_i)_i}_{\text{patch chain}}, \underbrace{\hat{\varepsilon}}_{\text{error estimate}} \right). \quad (1)$$

The physical key is $\text{hash}(x_{1:n})$ —tokens only. Adapter name and weight version are *not* in the key; they label edges of the lineage graph. This is the opposite of vLLM’s `lorax_name` extra hash and SGLang’s `weight_version` radix namespace.

Lookup lattice.

$$\text{lookup}(x_{1:n}, W_{\text{cur}}) = \begin{cases} \text{STRICT} & \text{if } v(C) = v_{\text{cur}}, \\ \text{PATCHED} & \text{if a } \Delta W \text{ path exists and } \hat{\varepsilon} \leq \varepsilon, \\ \text{MISS} & \text{otherwise (today’s recompute).} \end{cases} \quad (2)$$

Δ KV is *pointwise never worse* than vLLM/SGLang today: a miss is exactly flush-and-recompute, and a patch whose measured residual exceeds ε is promoted to a miss (the *correctness floor*).

Kill criterion. A patched cache is acceptable if (i) next-token KL($\pi_{\text{fresh}} \parallel \pi_{\text{patched}}$) $\lesssim 10^{-2}$ nats, (ii) max-layer relative KV $\ell_2 \lesssim 0.10$, and (iii) that ℓ_2 sits inside a 4-bit grouped-quantization noise floor (KIVI; Liu et al., 2024). KV- ℓ_2 alone is the wrong metric for serving and RL: sampling cares about π , and TIS/PPO care about $|\log \pi_{\text{fresh}} - \log \pi_{\text{patched}}|$.

Table 1: Compact ΔW payloads. PEFT convention throughout: $y = xW^\top + s(xA^\top)B^\top$, so per-token Δy lives in $\text{col}(B)$.

Regime	Payload	Rank
LoRA / adapter swap	PEFT $A \in \mathbb{R}^{r \times \text{in}}, B \in \mathbb{R}^{\text{out} \times r}, s = \alpha/r$	r (swap: $r_1 + r_2$)
ROME / MEMIT	outer product $uv^\top$	1
Quant swap	SVD sketch of $W_{\text{new}} - W_{\text{old}}$	chosen r
RL step	SVD sketch of a KL-bounded ΔW	chosen r

3 DELTA DESCRIPTORS

A weight update ships kilobytes, not a checkpoint.

Adapter swap is subtraction in factor space: $\Delta W_{A \rightarrow B} = \Delta W_B - \Delta W_A$, implemented by concatenating factors with a sign flip (rank adds). Quant and RL deltas that arrive dense are compressed once with an economy SVD to rank r and then treated identically to LoRA.

The *patchable regime* is the set of ΔW that are small by construction. If a Frobenius proxy $\|\Delta W\|$ exceeds a threshold (default 0.5), $\Delta K V$ refuses to patch: full SFT, architecture swaps, and other large moves are a miss. This is a feature, not a limitation—the lattice is allowed to say no.

4 EXACT PROJECTION PATCHES

Let $X \in \mathbb{R}^{n \times d}$ be the (RMSNORM’d) input of a k/v projection, and let $\Delta W_K, \Delta W_V$ be the corresponding maps. Then

$$\Delta K = \text{RoPE}(X \Delta W_K^\top), \quad \Delta V = X \Delta W_V^\top. \quad (3)$$

RoPE is a block-diagonal linear operator on each $(2i, 2i+1)$ pair, so

$$\text{RoPE}(K + \Delta K) = \text{RoPE}(K) + \text{RoPE}(\Delta K). \quad (4)$$

Equation equation 3 is therefore *algebraically exact* at any layer whose X is the true input under the *new* weights—in particular layer 0 when embeddings are frozen, and every layer whose residual-stream snapshot is stored (LMCache HiddenStateStore). For LoRA this is literally the adapter path, $O(ndr)$ vs. $O(nd^2)$ for the dense projection, and storage is $nr + rd$ codes+basis until RoPE mixes the K basis (after which we keep a dense increment of size $n d_{kv}$ —still one layer, still not a prefill).

On the reference stack, layer-0 K/V after equation 3 matches a fresh prefill to 2×10^{-5} absolute error (§7).

Above the first modified layer, X_ℓ itself depends on W , so equation 3 on a *stale* X_ℓ is a zeroth-order approximation—cheap, and the baseline every other route corrects.

5 DEEP-LAYER ROUTES

5.1 RANK- r SUBSPACE CORRECTION

AgentKVShift estimates a chunk-level offset $\hat{\mu}$ from a handful of recomputed “probe” tokens and adds it to everyone else. That is the rank-1 special case. Production LoRA is rank- r : ΔV lives in $\text{col}(B)$. Let $R = C_{\text{fresh}} - C_{\text{reused}} \in \mathbb{R}^{n \times d_{kv}}$ be a residual estimate (exact on probes; mixed-forward on the rest) and $B \in \mathbb{R}^{d_{kv} \times r}$ the LoRA B of v_{proj} . $\Delta K V$ solves, per token,

$$c_j = \arg \min_c \|R_{j,:} - Bc\|_2^2 = (B^\top B + \lambda I)^{-1} B^\top R_{j,:}^\top, \quad (5)$$

applies the structured increment $c_j^\top B^\top$, and mean-shifts only the leftover orthogonal to $\text{col}(B)$, estimated on probes:

$$\mu_\perp = \frac{1}{|\mathcal{P}|} \sum_{j \in \mathcal{P}} (R_{j,:} - c_j^\top B^\top). \quad (6)$$

Probes are overwritten with fresh values. Tokens whose residual still looks large relative to the probe scale are fully recomputed (CacheBlend residual gate; Yao et al., 2025).

After RoPE, ΔK is *not* in $\text{col}(B)$: rotation mixes coordinates per position. For K we therefore take the leading- r right singular vectors of the probe residual as the basis, at the same rank as the adapter.

The quality win over a scalar μ is per-token codes in the true column space of ΔW , not a collapse to rank 1.

5.2 FIRST-ORDER ANALYTIC PROPAGATION

When boundary activations $h_{\text{in}}, h_{\text{mid}}$ are stored, $\Delta K V$ propagates ΔX with closed-form Jacobians (derivation in Appendix A):

- **RMSNorm.** $y = x / \text{rms} \cdot w$; dy follows by differentiating $\text{rms} = \sqrt{\text{mean}(x^2) + \epsilon}$.
- **Attention.** $\Delta S = (\Delta Q K^\top + Q \Delta K^\top) / \sqrt{d_h}$, $\Delta A = J_{\text{softmax}}(A)[\Delta S]$ with the exact row-wise map $a \odot (\delta - \langle a, \delta \rangle)$, $\Delta \text{ctx} = \Delta A V + A \Delta V$.
- **SwiGLU.** $\text{d}(\text{silu}(g) \odot u) = (\text{silu}'(g) \odot dg) \odot u + \text{silu}(g) \odot du$.

Two stabilizers keep first-order error from compounding: (i) ΔX is rank-truncated to `propagator_rank` after every layer; (ii) ΔX is *reset to 0* every `hidden_stride` layers (default 4), after which that layer is exact-patched from the stored snapshot. The engine commits ΔW *before* propagating, so $dx W'^\top$ already contains the second-order $\Delta X \Delta W$ term.

When $\Delta Q, \Delta K, \Delta V$ are rank- r , forming ΔS is $O(n^2 r)$ not $O(n^2 d)$. On short sequences the dense softmax Jacobian still dominates, which is why analytic is accurate but not always cheaper than prefill (§7)—a kernel, not a math, gap.

5.3 κ -ANCHORED HYBRID ROUTING

Not every layer deserves the same work. Let $\kappa_\ell = \kappa(W_K^{(\ell)}) \kappa(W_V^{(\ell)})$ with $\kappa(W) = \sigma_{\max} / \sigma_{\min}$, and define the reverse cumulative

$$\tilde{\kappa}_\ell = \prod_{j=\ell}^{L-1} \kappa_j. \quad (7)$$

Shallow errors are amplified more (the LoRC observation). A log-normalized $\tilde{\kappa}$ plus a depth prior yields a *progressive probe budget*: the first third of the stack and high- $\tilde{\kappa}$ layers get up to 20% probes; deep, well-conditioned layers get 5%. The per-layer plan is then:

Condition	Strategy
No local ΔW and no upstream ΔW	skip
No local ΔW , upstream ΔX exists	subspace (correct ΔX -induced KV drift)
Stored X at layer 0 / every stride, k/v ΔW	exact, equation 3
Normalized $\tilde{\kappa} \geq 0.65$	probe (subspace + residual gate)
Otherwise	subspace (rank- r shift, no extra blend)

The default route is **hybrid**. Probe / hybrid patches are stored as the *full* patched-minus-base increment so a later STRICT replay does not drop the zeroth-order stage.

5.4 ERROR ACCOUNTING AND COST

Zeroth-order carries a Lipschitz compound bound so we never under-report: layer error is $e_\ell = L e_{\ell-1} + \|\Delta W_\ell\|$ with $L = 1.2$ by default. Probe and analytic may replace it with a measured residual. Chains add errors conservatively and rebase (rewrite the prefix as a new base) when $\hat{\epsilon} \geq 0.8 \epsilon$.

Approximate FLOPs:

$$c_{\text{prefill}} \approx L(nd^2 + n^2d + ndd_{\text{ff}}), \quad (8)$$

$$c_{\text{exact}} \approx (\#\text{LoRA maps}) \cdot ndr, \quad (9)$$

$$c_{\text{probe}} \approx \rho c_{\text{prefill}} + Lnd, \quad (10)$$

$$c_{\text{hybrid}} \approx c_{\text{exact}} + \frac{1}{2}c_{\text{probe}}, \quad (11)$$

$$c_{\text{analytic}} \approx L(ndr + n^2r). \quad (12)$$

The scheduler will not pick an `auto` route whose FLOPs exceed re-prefill. Combined with the ε floor, the engine is never more expensive *and* never less correct, pointwise, than flush.

6 SYSTEMS INTEGRATION

Δ KV is a library. The serving engines stay the serving engines.

vLLM. V1 loads external connectors from `kv_connector_module_path`. `DeltaKVConnector` duck-types `KVConnectorBase_V1`. On `load_lora_adapter / RL` weight broadcast the connector registers a ΔW instead of bumping a generation counter. It looks up the token-only prefix, materializes a patched view, and scatters it into already-allocated paged blocks $[N_{\text{blocks}}, \text{block_size}, n_{\text{kv}}, d_h]$. If ε is exceeded, `get_num_new_matched_tokens` returns 0 and vLLM prefills. The remaining upstream one-liner is: stop putting `lora_name` in the block hash when this connector is installed.

SGLang. Leave `--enable-weight-version-kv-isolation` off. Wrap `update_weights_from_tensor / update_weights_from_distributed` with `flush_cache=False`. `weight_version` is a *label on the delta*, not a radix key. In-flight requests keep private KV; the radix tree is patched in the background (`maintain.all`).

LMCache. Route A reads the existing `HiddenStateStore` (fp8/fp16 boundary activations). Chunk keys stay coupled to KV eviction; Δ KV does not grow a second store.

PEFT / HuggingFace. `from_peft_state_dict` accepts the usual `layers.i.self_attn.k_proj.loraA.weight` keys and PEFT scaling α/r . The adapter file is the delta descriptor.

Multi-LoRA shared prefixes. One physical row, keyed by the system-prompt tokens. Per adapter: a ΔW from base (or from the previously loaded adapter). N tenants share one prefix instead of N namespaced copies—the failure mode behind LMCache #2961 and vLLM same-name corruption.

7 EXPERIMENTS

We evaluate on the reference engine shipped with Δ KV: a Llama-style stack (RMSNorm, RoPE, GQA, SwiGLU) small enough to run on CPU, large enough that every algebraic claim is testable. This is a *kill-criterion* validator, not a substitute for a 7B/70B serving study. If the idea is algebraically wrong, it fails here; if it survives, the remaining work is wiring and kernels, not a different algorithm.

Protocol. Prefill under W_0 , commit a rank- r LoRA on k/v (and q where noted) of every layer, materialize a patched view, compare against a *fresh* prefill under $W_0 + \Delta W$. Metrics: max relative KV ℓ_2 , min cosine, next-token KL (nats), log-prob MAE, and the KIVI-4bit grouped-quantization floor of the fresh cache. Default toy: $L=4$, $d=64$, $n_{\text{heads}}=4$, $n=64$, rank 4, LoRA scale 10^{-2} .

What Table 2 says. (1) The adapter path on cached X already beats a 4-bit KV cache, at 3.4% of prefill. (2) The subspace/probe residual tightens ℓ_2 from 0.0035 to 0.0021 at a still-cheap 8–10% of prefill. (3) Analytic is numerically exact on this stack (first-order plus committed W' plus stride reset) but, on $n=64$, the dense softmax Jacobian makes it only a mild win—as predicted by the cost

Table 2: Kill-criterion validator. Rank-4 LoRA, scale 0.01, seq 64. All four routes sit inside the KIVI-4bit floor (0.066) with next-token KL = 0 at reported precision. Hybrid is the default.

Route	max rel. ℓ_2	KL (nats)	logprob MAE	within KIVI-4bit	FLOP ratio
Zeroth-order	0.0035	0	3.8×10^{-4}	yes	0.034
Analytic	0.0000	0	5.2×10^{-7}	yes	0.82
Probe	0.0021	0	1.8×10^{-4}	yes	0.10
Hybrid	0.0021	0	1.7×10^{-4}	yes	0.085

model. (4) Hybrid matches probe quality at lower billed FLOPs because boundary layers stay exact and only high- $\tilde{\kappa}$ layers pay the residual gate.

Layer-0 exactness. For a k/v -only LoRA on layer 0, equation 3 matches a fresh prefill to $< 2 \times 10^{-5}$ absolute error on both K and V , and cosine > 0.9999 . This is the algebraic claim, not a tuned approximation.

Subspace vs. rank-1 μ . On a synthetic residual $R = CB^\top + 1\mu_\perp^\top$ with varying codes C , per-token least-squares onto B recovers $\text{col}(B)$ and strictly beats a probe mean-shift (unit test `test_subspace_recovers_column_space_plus_orthogonal_mu`).

ROME. A rank-1 write $uv^\top$ into `down_proj` of layer 1 (scale 0.05) patches through the probe route; next-token KL remains defined and finite, and the engine does not flush. The delta descriptor is literally two vectors.

Correctness floor. A LoRA of scale 1.0 (outside the patchable regime) is a miss: the lattice refuses, and the engine prefills. A too-tight ϵ with a measured residual likewise falls back. These are the tests that ΔKV does not silently serve a stale cache.

Replay. After a hybrid materialize, a second lookup is `STRICT` and replays the stored patched-minus-base increment to numerical identity (10^{-5}). Storing only the probe increment on top of a zeroth-order view would drop a stage; the full increment is load-bearing.

Forty-eight unit tests cover RoPE linearity, RMSNorm / softmax / SwiGLU Jacobians, lattice decisions, PEFT parsing, adapter-swap subtraction, paged scatter/gather, and connector duck-types. GitHub CI runs the same suite on CPU PyTorch.

8 RELATED WORK

Prefix caches. PagedAttention (Kwon et al., 2023) and SGLang (Zheng et al., 2024) made token-prefix reuse the default. LMCash (LMCash Team, 2024) extends that reuse off-device and stores hidden states. All of them still namespace or flush when W moves. ΔKV reuses their storage and connectors; it changes the *invalidation policy*.

KV reuse under context shift. CacheBlend (Yao et al., 2025) selectively recomputes tokens whose layer-1 KV residual is large when stitching cached chunks. AgentKVShift estimates a chunk offset from probes. Both are *context-axis* techniques. ΔKV reuses the probe/gate machinery with a new trigger (weight delta) and a new structure (column space of B).

Low-rank weights. LoRA (Hu et al., 2022), QLoRA, and PEFT made $\Delta W = sBA$ the production adapter format. ROME (Meng et al., 2022) and MEMIT (Meng et al., 2023) are rank-1 edits. ΔKV treats those files as the delta descriptor rather than as a reason to drop the cache.

KV compression. KIVI (Liu et al., 2024), SnapKV, and related methods quantize or evict KV under a *fixed* W . We use KIVI’s 4-bit noise as a *floor*: a patch that sits inside that floor is in the noise of a serving optimization operators already ship.

Incremental view maintenance. The database analogy is deliberate (Gupta & Mumick, 1993). Materialized views are maintained under Δ relation; we maintain KV under ΔW . The lattice strict/patched/miss is the consistency contract.

9 LIMITATIONS

The reference validator is a 4-layer, $d=64$ CPU model. It can falsify the algebra; it cannot measure tail latency on Llama-3-70B. Error compounding through 32–80 layers, paged GPU kernels for the adapter path, and the one-line vLLM/SGLang hooks are remaining engineering work, not open research questions about *whether* $\Delta K = \text{RoPE}(X\Delta W_K^T)$. Analytic first-order still densifies ΔS in the PyTorch path; a low-rank attention kernel would change the flop ratio, not the Jacobian. $\Delta K V$ will not patch a full SFT or an architecture swap—by design. In-flight decode under a mid-token weight update is out of scope: live requests keep private KV; the radix/paged store is patched in the background.

10 CONCLUSION

Serving engines already know how to reuse KV along the token axis. They treat the weight axis as a cache apocalypse. That is unnecessary for the ΔW that actually occur in production: LoRA swaps, rank-1 edits, quant grid changes, KL-bounded RL steps. $\Delta K V$ makes the cache a materialized view, patches it with exact projection math where X is known and with a κ -anchored hybrid otherwise, and falls back to today’s recompute when it cannot. The kill-criterion holds on a real (if small) Transformer. The connectors fit engines that already exist. The next proof is a large-model serving run; the algorithm does not need to change for that proof to be well-defined.

REFERENCES

- Ashish Gupta and Inderpal Singh Mumick. Maintenance of materialized views: Problems, techniques, and applications. *IEEE Data Engineering Bulletin*, 16(2):3–18, 1993.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- LMCache Team. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. <https://docs.lmcache.ai>, 2024.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations*, 2023.
- SGLang Developers. Weight-version KV isolation in SGLang. Pull request #27886, <https://github.com/sgl-project/sglang/pull/27886>, 2025.
- Jiayi Yao, Hanchen Li, Yuhao Liu, Souradip Ray, Yihua Cheng, Qi Zhang, Kaidi Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, 2025.

378 Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao,
379 Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang:
380 Efficient execution of structured language model programs. In *Advances in Neural Information*
381 *Processing Systems*, 2024.
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431

A JACOBIANS

RMSNorm. Let $\text{ms} = \text{mean}(x^2)$, $\text{rms} = \sqrt{\text{ms} + \epsilon}$, $y = (x/\text{rms}) \odot w$. Then $\text{d}(\text{ms}) = 2 \text{mean}(x \odot \text{d}x)$, $\text{d}(\text{rms}) = \text{d}(\text{ms})/(2 \text{rms})$, and

$$\text{d}y = \frac{\text{d}x \cdot \text{rms} - x \cdot \text{d}(\text{rms})}{\text{rms}^2} \odot w. \quad (13)$$

Softmax. For a row $a = \text{softmax}(s)$, $J_{\text{softmax}}(a) = \text{diag}(a) - aa^\top$, so $J_{\text{softmax}}(a)[\delta] = a \odot (\delta - \langle a, \delta \rangle)$. This is exact-linear in δ ; the only approximation in attention is dropping higher-order terms in $(\Delta Q, \Delta K)$ when forming ΔS from stale Q, K —and, on the engine path, not even that, because Q, K are formed with committed W' .

SwiGLU. $\text{silu}(z) = z \sigma(z)$, $\text{silu}'(z) = \sigma(z) + z \sigma(z)(1 - \sigma(z))$. The MLP output differential is $\text{d}y = ((\text{silu}'(g) \odot \text{d}g) \odot u + \text{silu}(g) \odot \text{d}u) W_{\text{down}}^\top$ plus the local ΔW adapter path on the stale activation.

B WORKED LORA PATH

PEFT computes $\Delta y = s(xA^\top)B^\top$. Codes $c = sxA^\top \in \mathbb{R}^{n \times r}$ and basis B are the compressed $\Delta K/\Delta V$ before RoPE. After RoPE on K , the basis is position-dependent, so we store a dense $\Delta K \in \mathbb{R}^{n \times n_{\text{kv}} \times d_h}$. Compression relative to a full KV layer is d_{kv}/r before RoPE ($64\times$ at $d_{\text{kv}} = 4096$, $r = 64$). An adapter swap concatenates $(A_B, -A_A)$ and (B_B, B_A) , i.e. rank adds, without materializing ΔW .

C LOOKUP ALGORITHM

1. Hash the token prefix (no adapter, no weight version).
2. If no row: MISS, prefill, insert.
3. If row version equals W_{cur} : STRICT, replay base + $\sum$ patches.
4. If no ΔW path in the lineage DAG: MISS.
5. If $\|\Delta W\|$ proxy exceeds the patchable regime: MISS.
6. Pick a route (default hybrid) whose billed FLOPs are $<$ prefill.
7. Materialize. If the residual is *measured* and $\hat{\epsilon} > \epsilon$: MISS, prefill.
8. Else append the full patched-minus-base increment; rebase if $\hat{\epsilon} \geq 0.8\epsilon$.

D REPRODUCING THE VALIDATOR

The library is the paper’s reference implementation.

```
pip install -e ".[dev]"
python -m deltakv bench --seq 64 --rank 4 --scale 0.01
pytest
```

bench prints Table 2. Architecture notes live in docs/ARCHITECTURE.md; engine wiring in docs/INTEGRATION.md.